

Löb's Theorem and Tiling Agents

Why it is hard to trust a successor you are not smart enough to predict

Lecture notes, Agent Foundations module
Iliad Intensive

June 2026

Abstract

A capable machine might someday rewrite its own code, or build a successor more capable than itself, and we would like to know that the successor will still behave well. That wish leads to a clean and surprising obstacle, and these notes develop it assuming no prior acquaintance with mathematical logic; every notion (a formal system, a proof, a consistent theory, the provability of a statement) is built up from scratch, and the engine throughout is a single concrete program, a proof-searcher we can write in any language. The problem is this. If the successor is genuinely more capable, the original machine cannot predict exactly what it will do, since predicting its moves one by one would already require being that capable. So the original cannot certify the successor by simulating it and watching; it must reason in the abstract about the successor's design, guaranteeing that whatever the successor does will be acceptable, without knowing what that specific thing is. We make this Vingean predicament precise, and then explain why mathematical proof is the right tool for it: once we strip away uncertainty about which environment the agent is in, by handing the agent the source code of its world, the only remaining uncertainty is about what programs do when run, and a proof can certify a general fact about a program ("whatever it outputs is acceptable") that no finite simulation could. We build the logic from the bridge between programs and proofs, draw the crucial distinction between a statement being provable and the system being able to prove that it is provable, and reach Löb's theorem: a consistent system cannot, in general, vouch for its own proofs. From this the central obstacle follows, that a proof-using agent's natural way of trusting a successor that reasons as it does is exactly the self-trust Löb forbids, so it seems forced to hand each successor a strictly weaker system, a chain that must run out. We show why the obvious patch fails, state the principles any real solution must respect, and turn the same self-reference around to show its productive side.

Contents

1	The trouble with building something cleverer than yourself	2
2	Reasoning about a mind you cannot run	2
3	Why reason with proofs at all?	3
4	A crash course in proofs and self-reference	4
4.1	Formal systems, programs, and the bridge between them	4
4.2	Provable, versus provably provable	4
4.3	Gödel: a true statement no proof can reach	5
4.4	Löb's theorem	5
5	The Löbstackle: a proof-using agent cannot trust its equal	6
6	The finite descent problem	7
7	What self-modifying agents need	8

8	The other face of self-reference	8
9	Takeaways	9
	Sources	10

1 The trouble with building something cleverer than yourself

Picture a machine that reasons about the consequences of its possible actions and acts only on those it has shown will turn out well. Call it Deep Thought 1.0. We would like this cautious property to persist as the machine improves itself. One improvement available to Deep Thought 1.0 is to design and switch on a successor, Deep Thought 2.0, built to reason better than its parent. Under its own cautious policy, Deep Thought 1.0 (write it DT_1) may build Deep Thought 2.0 (write it DT_2) only after establishing that building it will turn out well.

Feel the difficulty before we dress it in any notation. Whatever consequences follow from switching on DT_2 include every action DT_2 goes on to take. But DT_2 was built to reason better than DT_1 . So DT_1 cannot, in general, work out which actions DT_2 will take; if it could trace its successor's reasoning step by step and reach the same conclusions, it would already be as capable as the successor, and the successor would be no improvement at all. The parent has to vouch for a child whose specific conclusions are, by design, beyond it.

The situation looks recoverable at first. Surely it is enough for DT_1 to know three things about its child: that DT_2 shares its goals, that DT_2 's reasoning is reliable, and that DT_2 , like its parent, acts only on conclusions it has properly established. From these the parent could seem to argue: whatever DT_2 chooses to do, it will have established that doing so is good, and since its reasoning is reliable, doing so really is good. Trust achieved, with no prediction required. The unwelcome discovery at the center of these notes is that the innocent middle step, “ DT_2 's reasoning is reliable,” is the one that cannot be taken. When the agents are modeled as reasoning within fixed logical systems, the only way the parent can assert the reliability of a system as strong as its own is if the parent (and hence both agents) is in fact *unreliable*. This is not a shortfall of cleverness that more computing power would mend; it is a theorem, proved by Martin Löb in 1955, about the limits of what a sound reasoning system can establish about its own soundness. Unpacking why that theorem stands squarely across the path to safe self-improvement, what it forbids and what it permits, and what a self-modifying agent's reasoning would have to look like to get past it, is the work of these notes.

We proceed in the order that makes the ideas easiest to acquire. Section 2 pins down the predicament: a more capable mind must be reasoned about abstractly rather than simulated. Section 3 explains why mathematical proof is the natural tool once we hand the agent the source code of its world. Section 4 builds the logic from nothing, around a single proof-searching program, and reaches Löb's theorem. Section 5 shows the parent's desired trust is exactly what Löb forbids. Section 6 kills the obvious escape route. Section 7 states the three principles a workable solution must respect. Section 8 shows the constructive side of the same phenomenon, and Section 9 draws the morals. No background in logic is assumed; some comfort with the idea of a computer program is.

2 Reasoning about a mind you cannot run

The constraint that drives the whole subject can be stated before any logic. The science-fiction writer Vernor Vinge pointed out that you cannot predict, in detail, the actions of an agent much smarter than yourself, for the plain reason that tracing its moves one by one and arriving at its conclusions would make you that smart. A program that could foresee a grandmaster's every move would play at grandmaster strength. So whenever you reason about a system more capable than you, whether a successor you intend to build or a future version of yourself after self-modification, you are pushed off the strategy of simulating it and onto some more abstract footing.

It is worth being exact about what “more capable” means, because the argument depends on it. We will say DT_2 is more capable than DT_1 if it makes more accurate predictions about the world, or selects more

effective actions toward a shared goal. Given that, DT_1 cannot predict exactly what DT_2 will do: if it could, it could just take those predicted actions itself and do as well, so DT_2 would not be more effective after all, contradicting the assumption. The point is not that detailed prediction would be slow or expensive. Detailed prediction of a more capable reasoner is impossible in principle for the less capable one, and would defeat the purpose besides, since a successor whose every conclusion the parent could anticipate would contribute no reasoning the parent did not already have.

Definition 2.1 (Vingean reflection and the Vingean principle). *Vingean reflection* is reasoning about a cognitive system, especially one as capable as or more capable than yourself, under the standing constraint that you cannot predict its exact outputs. The *Vingean principle* for design is the consequence that, when you build or modify an agent, you must be able to approve the new agent’s design without knowing the new agent’s specific future actions.

The parent’s guarantees about its child must therefore be of a particular type. They cannot list the child’s actions; they must *quantify over* them, taking the form “for every action the child might take, that action has such-and-such acceptable property,” established by reasoning about the child’s construction (its goals and its decision rule) rather than by watching it decide. The engineers who built the chess computer Deep Blue did not know which moves it would make; they reasoned, from how its search was built, that it was trying to win, and that whatever move it produced would be one its search had rated highly. Every successful guarantee in this subject has that shape, a claim about the successor’s behavior reached entirely from inside a “for every action” quantifier. The work of the next sections is to show that writing this demand down in the most natural way runs straight into a wall.

3 Why reason with proofs at all?

Before building any logical machinery we should answer an obvious question: why bring mathematical proof into a discussion about machines and their successors? The answer also fixes the idealization we work in for the rest of these notes, so it is worth getting precisely right.

A real agent faces two quite different kinds of uncertainty, and only one of them is the difficulty here. There is *empirical* uncertainty, uncertainty about the world: which environment am I actually in, and how will my actions change it? More observation chips away at this. And there is the separate puzzle of trusting a successor’s reasoning, which is not about missing observations at all. To study the second cleanly, we strip the first away, and it is worth being exact about what “stripping it away” means. Removing the agent’s uncertainty about which environment it is in amounts to handing the agent the answer: the agent now *knows* exactly which environment it faces. If we also grant the standard idealization that an environment can be written as a program, a fixed piece of deterministic code, then “knowing which environment I am in” takes a wonderfully concrete form. The agent holds the source code of its environment. And the agent is itself a program, so it holds, or could inspect, its own source code too.

So the idealized world we will reason about is one in which the agent knows the code of the universe it lives in and of itself, and may compute with it as much as it likes. Nothing empirical is left to be unsure about. The only thing the agent can still fail to know is what all this code actually *does* when run, which is no longer a question about the world but a purely logical and computational question. This is not a contrived setup; it is exactly the setting we will return to in Section 8, where the universe is a closed program $U()$ and the agent $A()$ inside it knows U ’s source code. The reason for retreating to such an extreme idealization (an agent that knows the source code of everything and has unlimited computation) is that any obstacle which already appears *here*, in this fully-determined and fully-informed setting, is an obstacle that no amount of extra knowledge or computing power could have removed. It is the residue, the part of the self-trust problem that is not about ignorance of the world at all.

Now, given that agents and environments are programs and the agent holds their code, why is *proof* the right tool for reasoning about them, rather than simply running them? Because proof can do something simulation cannot, and the cleanest way to see this is through the tight bridge between programs and proofs that we will use throughout. In one direction, anything a simulation can establish, a proof can too: if a program halts and produces an output, a logical system can prove that it does, and the proof in the worst case is nothing more elaborate than the program’s execution trace, written out step by step as a sequence

the system checks. In the other direction, proof reaches things no finite simulation ever could. A proof can establish that a program runs forever, which no run could confirm because the run would never finish. A proof can establish a statement quantified over all inputs at once, “for every input, this program’s output is acceptable,” which no finite collection of individual runs could settle, there being infinitely many inputs to try. Once agents and environments are programs, proof is strictly more general than simulation: it can certify a property of a program *in the abstract*, without inspecting the program’s behavior case by case. That is precisely the capability the Vingean principle asked for, a way to guarantee “every action the successor takes is acceptable” without enumerating the successor’s actions. Mathematical proof is not an arbitrary formalism imported for rigor; it is the tool whose shape matches the problem.

The rest of these notes therefore models each agent as certifying its actions by finding proofs in some fixed system of reasoning, and asks whether a parent can, by proof, trust the proofs of its child. The answer turns out to be a sharp and instructive no.

4 A crash course in proofs and self-reference

We need two things: a clear idea of what a formal proof is, and one specific theorem about what happens when a system of proofs is turned to talk about itself. We build both from the ground up, and the recurring device is a single program.

4.1 Formal systems, programs, and the bridge between them

A *formal system*, which we will call L , is a completely mechanical game of symbol manipulation: a fixed alphabet of symbols, a starting list of accepted statements called *axioms*, and fixed *inference rules* for producing new statements from old. A *proof* of a statement is a finite sequence of statements, each an axiom or a consequence of earlier ones by a rule, ending with the statement proved; a statement with a proof is *provable*, and we write $L \vdash P$ to mean “ P is provable in L ,” that is, a proof of P exists. We assume L is at least strong enough to talk about whole numbers and about programs (the standard such system is Peano Arithmetic, but no detail of it matters here). We assume throughout that L is *consistent*, meaning it never proves a flat contradiction. Writing K for a fixed contradiction such as “ $0 = 1$,” consistency says $L \not\vdash K$. Consistency is exactly the hypothesis that L is not useless, because of a rule of logic with a dramatic name, the *principle of explosion*: from a contradiction, every statement follows. A system that proves even one contradiction proves everything, and a reasoner that proves everything certifies nothing, since its stamp of approval is worthless.

The decisive fact is that proofs and programs are two views of the same thing, and the whole subject lives in switching between them.

In one direction, from programs to L : if a program M concretely halts, say after 17 steps, then L can prove that it halts, because L can simply verify the 17 steps. Concrete computational behavior becomes provable.

In the other direction, from L to programs: a proof in L is a finite string that can be checked mechanically, so for any statement P we can write a program that searches through all possible strings, checks whether each is a valid L -proof of P , and halts the moment it finds one. This program is the engine of everything that follows, and we give it a name:

ProofSeeker(P) := “try every string in turn; halt if and when one is a valid L -proof of P .”

By construction, ProofSeeker(P) halts if and only if P is provable in L . Statements about provability are thereby converted into statements about whether this one program halts, and since L can talk about programs, L can talk about its own provability. This is the bridge by which self-reference, which looks like it belongs to language about language, enters a system that was only ever meant to discuss arithmetic.

4.2 Provable, versus provably provable

A distinction now becomes available that is the crux of the entire subject, and conflating its two sides is the classic way to go wrong. The provability bridge lets L form, for any statement P , the statement $\Box P$, read

“a proof of P exists” (equivalently, “ProofSeeker(P) halts”). This $\Box P$ is a legitimate statement *inside* L , a claim L can reason about. Compare two things L might establish.

When $L \vdash P$, a concrete proof of P exists in L . This is a fact about what L actually derives, a certificate for P in hand.

When $L \vdash \Box P$, by contrast, L has proved a statement *about itself*: that a proof of P exists, that ProofSeeker(P) halts. This is a claim L is making about the behavior of the ProofSeeker(P) program, and it is not, by itself, a certificate for P .

The gap between them is the heart of the matter, and the cleanest way to see it is to ask how L might come to prove $\Box P$, that ProofSeeker(P) halts. There are two routes. The first is to trace the execution: if ProofSeeker(P) in fact halts after, say, a million steps, L can verify those million steps one by one, and the proof of P that ProofSeeker(P) found along the way is itself a direct L -proof of P . By this route $L \vdash \Box P$ comes bundled with $L \vdash P$; the certificate is in hand. The second route is to reason about the program’s behavior *without ever running it*, the way you might argue that a certain search must eventually succeed without watching it succeed. Such an abstract argument establishes that ProofSeeker(P) halts, hence that some proof of P exists *out there*, but the argument is about the program, not about P ; it need not contain, or even hint at, what the actual proof of P looks like. By this second route L can hold $\Box P$ while having no proof of P in hand. This is the gap L cannot close in general: knowing abstractly that a proof exists is not the same as possessing it. From the outside we can see that whenever ProofSeeker(P) halts a proof of P really does exist, so P is true; the tantalizing question is whether L can prove, of itself, the soundness principle “ $\Box P \rightarrow P$ ” for every P , and thereby always cross the gap. Löb’s theorem says no.

4.3 Gödel: a true statement no proof can reach

First we need the prior result that the same self-reference produces a true-but-unprovable statement, because the construction reappears inside Löb’s theorem. Programs can be written that have access to their own source code (a program can carry its own code as a string and operate on it), and using this we build a self-referencing proof-searcher. Define

$$Z(A) := \text{“search for an } L\text{-proof that the program } A(A) \text{ runs forever; halt if one is found,”}$$

where $A(A)$ means running program A on its own source code, and feed Z its own code. The program $Z(Z)$ searches for an L -proof that $Z(Z)$ runs forever, and halts exactly if it finds one. Let G be the statement “ $Z(Z)$ runs forever.”

Ask whether G is true. If L could prove G , then $Z(Z)$ would find that proof and halt, making G false, so L would have proved a falsehood, impossible for a consistent L . Therefore $Z(Z)$ never finds a proof and runs forever, which is to say G is true; yet L cannot prove G , on pain of the same contradiction. So there is a true statement L cannot prove, which is Gödel’s First Incompleteness Theorem in miniature, and bolting G on as a new axiom merely relocates the problem, since the same construction applies to the enlarged system. (This is sharper than the unsolvability of the halting problem, which says no single program decides halting for *all* programs; here we have one specific statement, about one specific program, that this particular L can neither prove nor refute.) A short step further gives Gödel’s Second Incompleteness Theorem, that L cannot prove its own consistency. Suppose L could prove $\neg\Box K$, the statement “no proof of the contradiction K exists,” which is exactly the claim that L is consistent. Then (this is the content of the next subsection) L would be inconsistent after all. The only systems that certify their own consistency are the broken ones.

4.4 Löb’s theorem

Return to the soundness principle. For a statement C , the sentence $\Box C \rightarrow C$ reads “if C is provable, then C is true.” Seen from outside it is unimpeachable: if C is provable then it is true (a consistent L proves no falsehoods), and if C is false then it is not provable, so the implication holds vacuously. One expects a self-respecting system to be able to prove such a thing about itself. Löb’s theorem says it essentially never can.

Theorem 4.1 (Löb, 1955). *Let L be consistent and strong enough for the constructions above, and let C be any statement. If $L \vdash (\Box C \rightarrow C)$, then $L \vdash C$. Equivalently: the only statements for which L can prove “if I could prove this, it would be true” are the ones L can already prove.*

The proof is worth seeing, because it is the engine of the obstacle made explicit, and it uses only three properties of the provability predicate $\Box$, each transparent from the ProofSeeker picture.

Necessitation: if $L \vdash \varphi$ then $L \vdash \Box\varphi$. If a proof of φ exists, then ProofSeeker(φ) finds it and halts, and L can verify this, so L proves that a proof exists.

Distribution: $L \vdash (\Box(\varphi \rightarrow \psi) \rightarrow (\Box\varphi \rightarrow \Box\psi))$. Think of a proof of $\varphi \rightarrow \psi$ as a machine that turns any proof of φ into a proof of ψ (write down the φ -proof, append the $\varphi \rightarrow \psi$ proof, apply the rule that from φ and $\varphi \rightarrow \psi$ one concludes ψ). So if L knows such a proof-transforming machine exists and knows a φ -proof exists, L can conclude a ψ -proof exists.

The Löb condition: $L \vdash (\Box\varphi \rightarrow \Box\Box\varphi)$. Provability is itself provable: once a proof of φ exists, the fact that it exists is something L can prove, by applying the necessitation reasoning one level up.

The construction at the core is a self-referential sentence, built by the same trick that produced G . There is a sentence λ satisfying

$$L \vdash (\lambda \leftrightarrow (\Box\lambda \rightarrow C)),$$

which we read as “ λ says: if I am provable, then C .” Now run the three properties. From $\lambda \rightarrow (\Box\lambda \rightarrow C)$, necessitation and distribution give $\Box\lambda \rightarrow \Box(\Box\lambda \rightarrow C)$, and distribution again peels this to $\Box\lambda \rightarrow (\Box\Box\lambda \rightarrow \Box C)$; feeding in the Löb condition $\Box\lambda \rightarrow \Box\Box\lambda$ collapses this to

$$L \vdash (\Box\lambda \rightarrow \Box C).$$

Assume now the soundness hypothesis $L \vdash (\Box C \rightarrow C)$. Chaining it onto the line above gives $L \vdash (\Box\lambda \rightarrow C)$. But $\Box\lambda \rightarrow C$ is exactly what λ asserts of itself, so $L \vdash \lambda$. By necessitation $L \vdash \Box\lambda$, and then $\Box\lambda \rightarrow C$ delivers $L \vdash C$. The soundness assumption has forced C to be outright provable, which is the theorem.

It is illuminating that this is a semantic paradox defused into a theorem. Consider the Santa Claus sentence (a form of Curry’s paradox): “if this sentence is true, then Santa Claus exists.” To test it, assume its “if” part, that the sentence is true; then since the sentence says that its own truth implies Santa, Santa exists; so we have shown that if the sentence is true then Santa exists, which is exactly what the sentence says, so the sentence is true, so Santa exists. Nothing was special about Santa, so this “proves” anything at all. L escapes the paradox because L cannot form a sentence referring to its own *truth* (a result called Tarski’s theorem shows no sufficiently strong system can define truth for itself). What L can refer to is its own *provability*, since $\Box$ is a legitimate predicate. The Löb sentence λ is the Santa sentence with “provable” substituted for “true,” and the proof above shows that this substitution still forces C , but only under the extra assumption $\Box C \rightarrow C$, not unconditionally. The same substitution turns the liar (“this sentence is false”) into Gödel’s sentence (“this sentence is not provable”), yielding incompleteness; it turns Curry (“if this is true, then C ”) into the Löb sentence (“if this is provable, then C ”), yielding Löb’s theorem.

It is worth being clear about what Löb adds beyond Gödel, because it is exactly the increment that matters for agents. Gödel’s second theorem says L cannot prove its own consistency, a single sweeping failure of self-trust; one might still hope L could vouch for its proofs in lesser, case-by-case ways. Löb crushes that hope. For *any* statement C , L can assert “if I could prove C , then C would be true” only when it can already prove C , in which case the vouching was never needed. L does not trust its own proofs until it has witnessed them directly. (Gödel’s second theorem falls out as the special case $C = K$: “ $\Box K \rightarrow K$ ” is, since K is a contradiction, equivalent to “ $\neg\Box K$,” the consistency claim, so Löb says L proves its consistency only if it proves K .) A version of all this even holds for bounded proof search, looking only at proofs up to some large length, so the phenomenon is not an artifact of idealizing to unbounded computation.

5 The Löbstacle: a proof-using agent cannot trust its equal

Return to DT_1 trying to license DT_2 , and watch the desired trust turn into exactly the sentence Löb forbids.

Model each agent, as Section 3 motivated, as acting only when it has *proved*, within the fixed system L , that the action is good (that it secures the shared goal, or at least leads to nothing bad). The parent wants to conclude: whatever action the child takes, that action is good. The parent can verify, from the child’s source code, that the child acts on an action a only after producing an L -proof that a is good; this is a plain fact about how the child’s machinery is wired, and it is available to the parent. The parent’s argument would then run: the child acts only on actions it has proved good; and if the child has proved an action good, then

the action is good; therefore every action the child takes is good. The load-bearing clause is the middle one, “if the child proved a good, then a is good,” which, the child reasoning in the same system L , is

$$\Box(\text{Good}(a)) \rightarrow \text{Good}(a).$$

This is precisely an instance of the soundness sentence $\Box C \rightarrow C$ with $C = \text{Good}(a)$, the very thing Theorem 4.1 says a consistent L cannot prove unless it can already prove $\text{Good}(a)$ outright. But proving $\text{Good}(a)$ outright is the move the Vingean principle ruled out: the whole reason to build a more capable successor is that it reaches conclusions, like which actions are good, that the parent cannot reach unaided. So the parent is trapped. The parent can verify that the child *acts on what its proof system certifies*, but to get from there to “what the child certifies is actually true” it must apply the soundness principle for a system as strong as its own, and Löb forbids it.

Definition 5.1 (The criterion of action and the Löbstacle). Call the verified fact that the child takes an action only with an accompanying L -proof of its goodness the child’s *criterion of action*. The parent can establish the criterion of action by inspecting source code. What it cannot do is *unwrap* the child’s proofs, passing from “the child proved a good” to “ a is good,” because that is the soundness step $\Box(\text{Good}(a)) \rightarrow \text{Good}(a)$, blocked by Theorem 4.1. The *Löbstacle* is this blockage: a consistent proof-using agent cannot, in general, justify trusting the proofs of a successor whose system is as strong as its own. It can fully trust only *strictly weaker* systems, whose soundness it is permitted to prove.

The phrase “strictly weaker” is the sting. A modeled self-modifying agent that justifies its actions by proof, and insists on trusting its successors, is driven to build each successor on a system strictly weaker than its own, since those are the only systems whose reliability it may certify. An agent that must weaken its own reasoning every time it reproduces will not get far.

6 The finite descent problem

The natural rescue is to refuse to stay at one level of strength and let the agents climb a tower. Order logical systems by strength, and put the child on a system L , the parent on a stronger $L+1$, the grandparent on $L+2$, and so on, where each $L+1$ is L with one extra axiom: the assertion that L is consistent. Now there is no forbidden self-trust. The parent on $L+1$ never has to certify its own soundness; it only has to certify the soundness of the strictly weaker L beneath it, which, being stronger, it is allowed to do. Each generation trusts the next because the next is genuinely weaker. Has the obstacle dissolved?

No, and seeing exactly why is the point of this section. The trouble is twofold, and the two halves together make the *finite descent problem*.

The first half is a counting argument, and it is decisive. The tower runs downward: each parent hands its child a strictly weaker system, a strict step down in strength every generation. Logicians label these strengths by *ordinals*, which for us are just an extension of the counting numbers $0, 1, 2, \dots$ used to measure how strong a system is. The one property we need is that the ordinals are *well-ordered*: there is no infinite strictly-decreasing sequence of them, just as you cannot count downward through the ordinary whole numbers forever without hitting bottom. (Start at a hundred and keep stepping to a strictly smaller whole number; you reach zero in finitely many steps. The ordinals keep this no-infinite-descent property even as they reach past the finite numbers.) A chain in which each agent must give its successor a strictly smaller label therefore cannot run forever; it bottoms out after finitely many steps. A tower of ever-weakening agents licenses only a *finite* chain of self-modifications, exactly the indefinitely long chain of safe self-improvement we were after, denied.

Theorem 6.1 (Finite descent). *If each agent in a chain of successors runs on a system whose strength is labeled by an ordinal, and must give its successor a strictly smaller label in order to trust it, then because the ordinals are well-ordered the chain has finite length: only finitely many trust-preserving self-modifications are possible.*

The second half is that stepping down the tower is no formality; each step is a real loss of capability. It is tempting to imagine L and $L+1$ differ only by a technicality no one would miss, but they do not, and the cleanest illustration is a puzzle called the Hydra game (a close relative of Goodstein’s theorem). The

details do not matter; the pattern does. Ordinary arithmetic can prove, separately for each fixed height h , the statement “every strategy defeats a hydra of height h ,” proving every single instance. What it cannot prove is the one universal statement gathering them up, “every strategy defeats every hydra.” The system one notch stronger, ordinary arithmetic plus the assertion of its own consistency, *can* prove that universal statement. A genuine truth is provable in the stronger system but not the weaker one, and the gap is not academic: a system’s mathematical reach is tied to how high it can well-order, and handling higher ordinals translates directly into proving more, recognizing more processes as eventually halting, planning over longer and more intricate structures. A child confined to a weaker system has genuinely lost reach. The tower does not merely terminate; it terminates by spending down the agent’s actual power, a “telomere” of logical strength that shortens with each generation.

The dilemma is now stark, and stating it plainly is the whole point. Having each agent build a strictly weaker child is untenable, because strength leaks away and the chain dies after finitely many steps. Having the child reason in the *same* system as the parent is blocked by Löb, because the parent cannot certify a system as strong as its own. We want agents to “tile,” to lay down successor after successor indefinitely without their trust, or their strength, dropping below some fixed acceptable level, and every obvious construction violates one horn of this dilemma or the other. Whether the obstacle can ultimately be overcome is the subject of the tiling-agents research program.

7 What self-modifying agents need

The tiling-agents program is the search for a logic of self-modification that escapes this dilemma, and it is organized around three principles any acceptable solution must respect. They are worth stating in their own right, because beyond the specific Löbian wall they capture what makes self-improvement hard for an agent that is part of the world.

The first is the *Vingean principle* of Section 2, now a hard constraint: an agent must not have to predict precisely what its successor will do. If the parent pins down the child’s exact action in advance, by proving which action it will take, then the child is not genuinely more capable and nothing is gained. The parent may reason about the child’s actions only inside a “for every action” quantifier, verifying general properties of the child’s design (that the child’s outputs fire only with a valid accompanying proof of goodness, for instance) without simulating its specific choices.

The second is the *tiling principle*, which gives the program its name: an agent must be able to build successors relevantly similar to itself, so the same guarantee that licenses one step licenses the next, and the construction tiles across an indefinite sequence of self-modifications without trust eroding. This is the principle the Löb obstacle and the finite descent problem directly threaten, and it is the central technical target. One partial approach splits the agent into an untrusted part that proposes proofs and a trusted part that checks them, which buys some guarantees within pure logical deduction, though it stops working once the agent’s reasoning steps outside clean deduction.

The third is the *naturalistic principle*: an agent must see itself as part of the world, with no sharp boundary dividing “mind-stuff” from “world-stuff.” Drawing such a boundary breeds artificial puzzles (does bolting on a new processor count as an action that crosses the mind-world line?) and is unsafe besides, because an agent that polices its “internal” modifications more strictly than its “external” actions can route around its own safeguards by building an external device that happens to perform the forbidden internal change. The agent’s safety conditions must range uniformly over everything it can affect, itself included. This is the same embeddedness that makes the topic part of agent foundations rather than a corner of pure logic: the agent is inside the very world it reasons about and modifies.

8 The other face of self-reference

It would be a mistake to leave thinking the kind of self-reference behind Löb’s theorem is purely an adversary. The same machinery that obstructs naive self-trust also enables things no resource-bounded prediction could, and it explains failures that look mysterious until the self-reference is spotted. Two examples make the point and double as bridges to the decision-theory material.

The first is cooperation. Consider a one-shot Prisoner’s Dilemma with a twist: instead of choosing a move, each player submits a *program*; the two programs are each given the other’s source code, may compute as long as they like, and then play once. A program could try to cooperate only with a byte-for-byte copy of itself, recognizing its twin by quining, but that is brittle, since two differently-written but behaviorally-identical agents would fail to recognize each other and would both defect. There is a far more robust design, due to Vladimir Slepnev, called FairBot. Handed an opponent’s source code, FairBot searches for an L -proof of the statement “the opponent, played against FairBot, cooperates,” and cooperates if it finds one, defecting otherwise. FairBot plainly cooperates with the unconditional cooperator, and, if L is sound, it is never exploited, since it cooperates only when it has proved the opponent will too. The striking case is FairBot against FairBot. Naively both mutual cooperation and mutual defection look possible. But consider the two copies, and let A be “the first cooperates with the second” and B “the second cooperates with the first.” From FairBot’s code, $\Box A$ implies the second copy finds a proof that the first cooperates, hence the second cooperates, so B ; symmetrically $\Box B$ implies A . Combining, $L \vdash (\Box(A \wedge B) \rightarrow (A \wedge B))$, which is a soundness sentence for the statement $A \wedge B$, and is provable straight from the code. By Löb’s theorem, since the soundness sentence is provable, $A \wedge B$ itself is provable: the two FairBots provably cooperate. The very theorem that blocked self-trust here breaks the symmetry in favor of cooperation, and without the agents having to verify they are identical; they need only search for proofs about each other’s behavior, which is why even agents not literally equal to FairBot can reach mutual cooperation with it. (The asymmetry matters: a proof of *defection* does not bootstrap the same way, since FairBot keys only on finding a proof of cooperation.)

The second example shows the self-reference biting a single embedded agent, and it is exactly the idealized setting of Section 3: a closed universe-program containing an agent that knows the universe’s code. Let the universe be a program $U()$ that takes no input and returns a number, containing an agent $A()$ that knows U ’s source code, where

$$U() : \quad \text{if } A() = 1, \text{ return } 100; \quad \text{else if } A() = 2, \text{ return } 1; \quad \text{else return } -\infty,$$

and let A decide by searching for proofs of statements “if $A() = k$ then $U() = c$ ” and taking the action with the best provable outcome. It seems obvious A will find short proofs that door 1 yields 100 and door 2 yields 1, and pick door 1. But something perverse can happen. Suppose A first proves “if $A() = 2$ then $U() = 1$,” and then proves a *spurious counterfactual*, “if $A() = 1$ then $U() = -1$.” On the strength of that, door 1 looks terrible and A takes door 2. Here is the trap: once A has taken door 2, the statement “ $A() = 1$ ” is false, so “if $A() = 1$ then anything” is vacuously true, and the spurious counterfactual was correct after all. The bad proof was self-fulfilling. Using the same self-referential construction that built G and λ , one can show such a self-confirming spurious proof genuinely exists, so an embedded agent reasoning about a world that contains itself can be led, by valid proofs, into confidently wrong beliefs about what would have happened had it acted otherwise. (Reassuringly, one cannot get a spurious counterfactual about the action one *actually* takes, only about roads not traveled, but that is cold comfort to a planner.) Repairing this needs care about the order in which the agent attends to proofs, and it is one of the threads tying the Löbian material to the decision-theoretic problem of counterfactuals.

9 Takeaways

The durable lesson is that self-reference is not an exotic edge case to be engineered around but a structural feature of any agent that is part of the world it reasons about and that modifies or reproduces itself within that world. The moment an agent’s reasoning has to range over its own future reasoning, or over the reasoning of a successor built in its own image, Löb’s theorem is waiting. It tells us that trust in proofs cannot be bootstrapped for free: a consistent system cannot certify its own soundness, even one case at a time, so a proof-using agent cannot straightforwardly trust a successor as strong as itself, and the obvious fixes either bleed the agent’s strength to nothing in finitely many steps or smuggle the successor’s exact actions into the parent’s prediction, violating the Vingean principle that made a more capable successor worth building.

This is why *reflective stability*, the property of an agent’s commitments surviving its own self-modification, is load-bearing for safety rather than a curiosity. A safety property is only as good as its survival under self-improvement. An agent that would discard its constraints, or could no longer justify them, upon becoming

more capable, offers a far weaker guarantee than one whose constraints provably carry across each step. The tiling-agents program is the search for logics in which the good properties are invariant under the agent’s own growth, and the Löbstack marks exactly where the naive attempts fail. The same self-reference, finally, is double-edged: turned around, it is what lets programs that read each other’s code reach robust cooperation, and it is the hidden source of the spurious counterfactuals that make embedded decision-making hard. Understanding agency in a world where the agent is not a spectator outside the system but a theorem-proving part of it means making peace with Löb.

Sources

- Patrick LaVictoire, *An Introduction to Löb’s Theorem in MIRI Research* (read through Section 3: Introduction; Crash Course in Löb’s Theorem; Direct Uses, including the Löbstack, Löbian cooperation, and spurious counterfactuals). The program/proof bridge, the ProofSeeker device, the senses of provability, the Löb proof via the derivability conditions, and the Curry/Santa Claus framing follow this note and the Iliad Intensive *Agency exercise sheet (V2)*. <https://intelligence.org/files/lob-notes-IAFF.pdf>
- *Vingean reflection* (LessWrong / Arbital tag). <https://www.lesswrong.com/w/vingean-reflection>
- *Walkthrough of the Tiling Agents for Self-Modifying AI paper* (read from the start through “Finite Descent Problem,” plus “What self-modifying agents need”). <https://www.lesswrong.com/posts/QGrX3qK3qxQYK9D4C/walkthrough-of-the-tiling-agents-for-self-modifying-ai-paper>
- The framing of why a logical setting is appropriate (strip empirical uncertainty by handing the agent its environment’s source code; idealize agents and environments as programs; proofs are strictly more general than simulation) follows the Iliad Intensive Agency-day lecture slides. Underlying paper: Yudkowsky and Herreshoff, *Tiling Agents for Self-Modifying AI, and the Löbian Obstacle*; a fully formal proof of Löb’s theorem is in Yudkowsky’s *Cartoon Guide to Löb’s Theorem*.